



REINFORCEMENT LEARNING

< START TREK />



REINFORCEMENT LEARNING



binary name: *.py
language: Python
compilation: None
authorized libraries: gymnasium, pytorch
additional documents needed: Anything used as a support during your defense

Start the Trek through the stars

You are part of one of the first teams exploring our solar system.

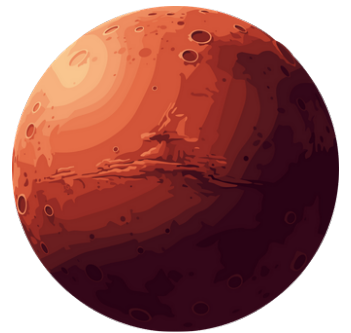
Destination: Mars.

However, to carry humans safely across space, we must first set up a relay base with resources on the Moon.

While the long-term objective is to be able to launch manned missions, you can't afford to put a pilot in every lunar module just yet.

Thus, **you will have to train the module to land by itself.**

Your role is to train a module in a simulator to do just that.



You will program an algorithm that enables the module to correct its own trajectory. It will be a reinforcement-learning agent that learns by trial and error from sparse rewards (soft penalties per step, strong bonus on safe landing).

You will emphasize the RL loop (state → action → reward), exploration–exploitation trade-offs, and basic value/Q estimation before any deep networks.

Success is measured by consistent autonomous landings and improved sample-efficiency across iterations.

You will build disciplined engineering habits (metrics, reproducibility, readable code).

Then—before sending a multi-million-dollar prototype into space—you will deliver a detailed report, provide evidence of your tests, explain your modifications, and defend, with structural and functional arguments, why your solution is the best.

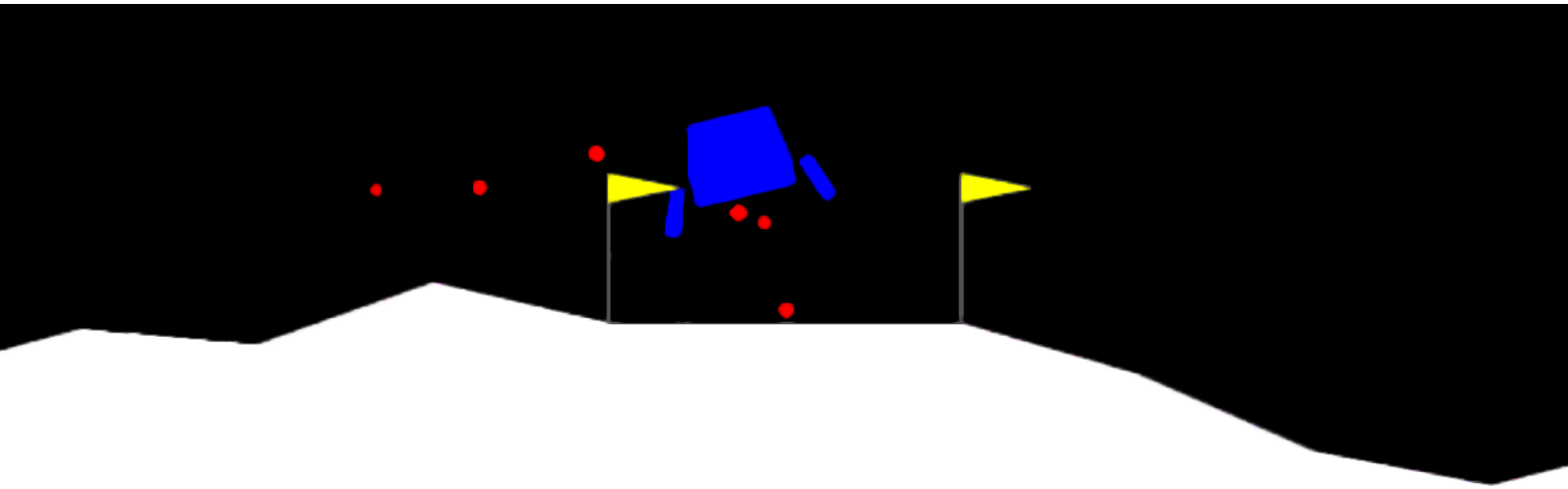


Shoot for the moon, and hopefully you'll land on the moon

Lunar Lander

Gymnasium

In this project, we will use **Gymnasium**, an API standard for reinforcement learning with a diverse collection of reference environments. The environment you'll use is [Lunar Lander](#) : a 2D rocket landing task built on Box2D physics. It will take care of the inner workings of the lunar module, its representation, the physics, as well as a lot of other interesting features, so that you can focus on the reinforcement learning part. The code you'll produce will be in python.



Parameters

Actions

Discrete version:

- do nothing
- fire left thruster
- fire main engine
- fire right thruster



A continuous-control version also exists, you are free to experiment with it.

State

The state is an 8-dimensional vector:

x: horizontal position

y: vertical position

v_x: horizontal velocity

v_y: vertical velocity

theta: lander angle

theta_dot: angular velocity

left_leg_contact: 1 if the left leg is in contact with the ground, else 0

right_leg_contact: 1 if the right leg is in contact with the ground, else 0

Reward

The reward function ("reward shaping") encourages:

- being close to the landing pad
- moving slowly
- staying level (lander kept horizontal)
- landing with both legs

There are also:

- per-step penalties when engines are firing
- about +100 for a safe landing
- about -100 for a crash



The environment is considered solved when the mean score is greater than or equal to 200 (without wind which is considered a bonus)

Usage (Gymnasium)

```
import gymnasium as gym
env = gym.make("LunarLander-v3", render_mode="rgb_array") # discrete by default
obs, info = env.reset(seed=SEED)
while True:
    action = policy(obs) # random or learned
    obs, r, terminated, truncated, info = env.step(action)
    if terminated or truncated:
        obs, info = env.reset()
```

For **continuous control** or wind:

```
gym.make("LunarLander-v3", continuous=True, enable_wind=True, wind_power=15.0, turbulence_power=1.5).
```

Goals

Deliverables

- ✓ **Codebase** (clean repo; `train.py`, `eval.py`; README).
- ✓ **YAML configs** for each run (env, algo, hypers, seeds).
- ✓ **Tracking & assets**: returns/episode length plots, termination cause stats, **videos**.
- ✓ **Short report** (4-6 pages): setup, results, ≥ 1 ablation, limits, next steps.
- ✓ **One-click repro** script to regenerate figures + CSV.

Tasks

1. Baseline & instrumentation

- ✓ Random policy and a simple heuristic; log returns, episode length, **why** it ended (crash/out-of-view/sleep). Save videos.

2. Agent (discrete)

- ✓ MLP (2–3×ReLU), replay, target net, epsilon-greedy decay. Plot returns, epsilon, loss.

3. Ablation (≥ 1)

- ✓ e.g., buffer size; hard vs soft target updates; observation normalization; reward clipping.

4. Reproducibility

- ✓ **≥ 5 seeds** (0..4), report mean $\pm$ 95% CI for key metrics.

5. Defense

- ✓ You must be able to justify your choices, benchmark different techniques and talk about your reasoning

Acceptance criteria

- ✓ Achieve **mean score ≥ 200** over **100 consecutive episodes** (report distribution & variance).
- ✓ Correctly handle and **log** `terminated` vs `truncated` and the termination reason (crash, out-of-view, sleep).
- ✓ One-click repro regenerates the same plots/CSV within $\pm 5\%$ with the same seeds/configs.

v0.0.1

{EPITECH}